

PrediTeq

Maintenance predictive intelligente pour un stockeur vertical rotatif

Projet PFE: Machine Learning + IoT + application web
Du signal machine a la decision de maintenance

Simulation

ML

Runtime

Web app



AROTeq

Soutenance jury | Version generee depuis les artefacts verifiees du depot



Le probleme industriel

Pourquoi PrediTeq existe et a quoi il sert



Intervenir trop tard peut mener a une panne ou a un arret.

Intervenir trop tot cree un cout de maintenance inutile.

Il faut donc surveiller l'etat de sante et anticiper le

Deux sorties cle

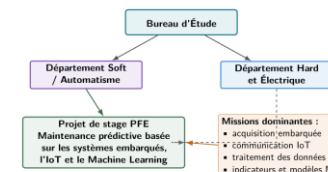
Health Index = note de sante entre 0 et 1. RUL = temps restant avant la zone critique.

Cas cible

Stockeur vertical rotatif industriel AROTEQ utilise dans un contexte de production.

Valeur metier

Aider a prioriser la maintenance, planifier et reduire les interventions mal synchronisees.



Notre demarche methodologique

Donnees limitees au depart, progression par phases, collecte reelle en parallele

1. Partir du reel

Le projet part d'une vraie machine, mais sans longues historiques de pannes annotees au debut. Il fallait donc avancer sans attendre des annees.

2. Simuler de facon realiste

Nous avons construit un socle d'entrainement a partir du moteur, du cycle, des charges, du bruit capteur et d'un contexte plausible.

3. Avancer par phases

Nous n'avons pas tout modele d'un coup : signaux, indicateurs, anomalies, note de sante, puis seulement temps restant.

4. Valider chaque bloc

La chaine est verifiee sur un test separe, sur des trajectoires completes, sur NASA CMAPSS et par calibration. NASA sert ici de repere externe, pas de donnees principales du stockeur.

5. Collecter le reel en parallele

Le flux LabVIEW / PLC vers le backend, l'historisation cloud et le journal maintenance preparent deja la phase suivante : raffiner et annoter avec du vrai terrain.

Le choix des variables vient aussi du terrain : plaque moteur, cycle reel et logique charge -> puissance -> courant -> echauffement discutee avec le technicien.

Les données d'entrainement simulees

Ce que contient vraiment le jeu cree a l'etape 1

Une trajectoire = l'histoire complete d'une machine simulee dans le temps.

Chaque trajectoire combine un profil de degradation et un cas de charge.

Le script exporte ensuite les signaux utiles pour tout le pipeline ML.

200

trajectoires

4

profils

20

cas de charge

Colonnes cle

trajectory_id, profile, load_kg, t_seconds, rms_mms, power_kw, current_a, temp_c, humidity_rh, phase, plus des metadonnees internes de simulation.

Sortie du script: prediteq_ml/data/raw/trajectories.csv

Preuve 7 - Jeu d'entrainement simule

200 trajectoires, 4 profils, 20 cas de charge et colonnes exportees vers trajectories.csv

```
prediteq_ml/steps/step1_simulate.py
# Etape 1 - Simulation de données
# Génère N_TRAJECTOIRES (=200) trajectoires de dégradation physiquement contraintes.
# Chaque trajectoire a un cas de charge qui détermine l'appel de courant.
# Sortie : data/raw/trajectories.csv

N_TRAJECTOIRES = 200
N_PROFILES = 4
N_LOAD_CASES = 20

# Chaîne de dégradation (d'après technicien Arnetes, confirmée par IEC 60034-1 §4.5) :
# charge P = puissance P = courant I * échauffement bobines (loi I^2 R)
# vieillissement isolation (Arrhenius) = fatigue siliers (Lundberg-Palngren)
# MS = RMS vibratoire ? (ISO 2818-3)
# La variable électrique de régression est le COURANT : tension (400V) et
# vitesse (1510 RPM) sont constantes d'après la plaque signalétique.

# Profil de dégradation (familles standard en MM, cf. Savane & Siebel 2008) :
# A_linear : dégradation uniforme, usure abrasive simple
# B_quadratic : profil à deux temps (lent puis accéléré), formule M(t)
# C_stepwise : défauts successifs discrets (dents d'engrenage, écaillage)
# D_noisy_linear : linéaire + bruit mesure (cas réaliste avec capteurs bruités)

...
return pd.DataFrame({
    'trajectory_id': traj_id,
    'profile': profile,
    'load_kg': load_kg,
    't_seconds': t_seconds,
    'rms_mms': rms_mms,
    'power_kw': power_kw,
    'current_a': current_a,
    'temp_c': temp_c,
    'humidity_rh': humid_rh,
    'simulated_h': h1,
    'phase': phase,
})

...
if __name__ == '__main__':
    all_dfs = []
    traj_id = 0

    n_each = N_TRAJECTOIRES // N_PROFILES * N_LOAD_CASES // N_LOAD_CASES

    # Répartir les cas de charge entre les trajectoires de chaque profil :
    # 20 cas de charge = 2 trajectoires par cas = 10 cas répétés (50 = 20*2*10)
    load_list = LOAD_CASES_KG * (n_each // N_LOAD_CASES) * \
        LOAD_CASES_KG[n_each % N_LOAD_CASES]
    load_list = sorted(load_list) # ordre déterministe

    for profile in PROFILE_NAMES:
        loads_for_profile = load_list[n_each]
        print(f" Simulation (profile) {(n_each) trajectoires, "
              f"Charges: (sorted(set(loads_for_profile))) kg...")
        for i in range(n_each):
            load_kg = loads_for_profile[i]
            df = simulate_trajectory(traj_id, profile, load_kg)
            all_dfs.append(df)
            traj_id += 1

    trajectories = pd.concat(all_dfs, ignore_index=True)

    out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'data', 'raw', 'trajectories.csv')
    out_path = os.path.abspath(out_path)
    os.makedirs(os.path.dirname(out_path), exist_ok=True)
    trajectories.to_csv(out_path, index=False)

    print(f"VOK: Etape 1 terminée - {traj_id} trajectoires, {len(trajectories)} lignes")

# Capture générée automatiquement depuis le code source réel du dépôt PrediTeq
```

prediteq_ml/steps/step1_simulate.py

Pourquoi ces variables et comment elles donnent ces resultats

Le choix des variables suit une chaine de cause a effet lisible

Charge

plus la charge est lourde, plus la montee demande d'effort

>

Effort electrique

la puissance monte, donc le courant monte aussi

>

Vieillessement

l'echauffement et l'usure font baisser le Health Index

>

Signaux visibles

vibration, puissance, courant, temperature et humidite evoluent

Pourquoi ces variables ?

Nous avons garde des variables soit mesurables en live, soit directement derivees du comportement machine. Ainsi, l'entrainement offline et le runtime utilisent le meme langage physique.

Message non technique

Nous n'avons pas choisi des colonnes arbitraires. Nous avons choisi les mesures qui changent quand la machine travaille plus fort ou se degrade.

Preuve 1 - Constantes moteur, charge et cycle

Capture du code source reel de la configuration de simulation

```
prediteq_ml/config.py
12 # Note : le technicien a dit « 380V » à l'oral - ancien nom pour 400V (harmonisation UE).
13 # La plaque signalétique indique 400V. On utilise la valeur de la plaque.
14 MOTOR_SPEED_RPM = 1410 # Constante - ne varie pas avec la charge
15 MOTOR_POWER_KW = 2.2 # Puissance utile sur l'arbre (mécanique)
16 MOTOR_VOLTAGE_V = 400 # Tension triphasée (couplage Y, d'après plaque)
17 MOTOR_COSPHI = 0.80 # Facteur de puissance (d'après plaque)
18 REDUCER_RATIO = 1/60
...
26 # Géométrie machine
27 N_FLOORS = 19
28 LOAD_PER_FLOOR_KG = 15 # Max 15 kg par étage (d'après technicien)
29 LOAD_MAX_KG = 19 * 15 # = 285 kg - 19 étages entièrement chargés
30 LOAD_NOMINAL_KG = 10 * 15 # = 150 kg - demi-charge (fonctionnement typique)
...
38 # Timing du cycle (secondes)
39 T_ASCENT_S = 12
40 T_DESCENT_S = 12
41 T_PAUSE_S = 20
42 T_CYCLE_S = 44
...
48 # N_TRAJECTORIES = 200 : dimensionnement statistique
49 # - 50 trajectoires par profil x 4 profils couvre toute la plage de charge
50 # (20 cas 0-285 kg, répétés ~2.5x) et stabilise la variance CV
51 # - Ordre de grandeur similaire à CHAPSS FDB02 (100 unités) et FEMTO-ST PRONOSTIA
52 # (17 roulements run-to-failure), Saxena & Goebel 2008
53 N_TRAJECTORIES = 200
```

prediteq_ml/config.py

Preuve 2 - Puissance, phase et courant

Le cycle ascent / descent / pause et la derive electrique sont simules dans le code

```
prediteq_ml/steps/step1_simulate.py
146 for i in range(n):
147     pt = phase_t[i]
148     h = hi[i]
149     if pt < T_ASCENT_S:
150         phase[i] = 'ascent'
151         # Puissance = base (friction) + composante charge + composante dégradation
152         # P = P_vide + P_plage_charge * (charge/max) + P_plage_deg * (1 - HI)
153         p_load = P_ASCENT_EMPTY_KW + P_ASCENT_LOAD_RANGE * load_ratio
154         p_deg = P_ASCENT_DEG_RANGE * (1 - h)
155         power[i] = np.clip(p_load + p_deg, P_ASCENT_EMPTY_KW, P_ASCENT_DEG_KW)
156     elif pt < T_ASCENT_S + T_DESCENT_S:
157         phase[i] = 'descent'
158         power[i] = P_DESCENT_KW
159     else:
160         phase[i] = 'pause'
161         power[i] = P_PAUSE_KW
162
163 # Bruit capteur (PAC2200 - fiche technique section 3.1) ; xnoise_mult
164 # pour capteurs dégradés (profil D, simule EMI variateur).
165 power += np.random.normal(0, NOISE_PAC2200 * P_ASCENT_NOM_KW * noise_mult, size=n)
166 power = np.clip(power, 0.0, 3.0)
167
168 # Courant : I = P / (√3 × V × cosφ) - tension constante à 400V
169 current = (power * 1000) / (MOTOR_SQRT3_V_COSPHI * KW - W, puis / dénominateur
```

prediteq_ml/steps/step1_simulate.py

La logique des 3 machines simulees

Meme base projet, mais 3 contextes d'usage pour montrer 3 etats de sante

ASC-A1 - machine protegee

Bizerte | profil A_linear | charge 0-85 kg | cible HI 0.96. Usage léger, environnement plus sec, peu de surcharge.

ASC-B2 - machine sous surveillance

Sfax | profil C_stepwise | charge 70-190 kg | cible HI 0.62. Trafic mixte, demi-charges frequentes, stress moyen.

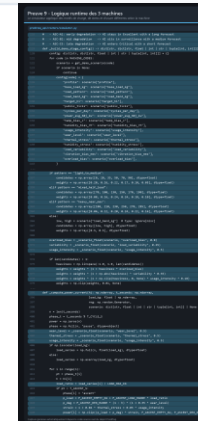
ASC-C3 - machine critique

Sousse | profil D_noisy_linear | charge 180-285 kg | cible HI 0.10. Ligne intensive, charges lourdes, usure et stress eleves.

Message simple: ce ne sont pas 3 graines aleatoires. Le simulateur change vraiment les charges, le stress, l'usure et le point de depart.



prediteq_api/demo_scenarios.py



prediteq_api/routers/simulator.py

Pipeline ML en langage simple

Chaque bloc a un role clair, puis le backend charge les artefacts exportes



Lecture simple pour le jury

Nous avons avance par blocs: simulation -> indicateurs -> anomalies -> HI -> RUL.

Chaque bloc a ete valide avant le suivant, puis exporte vers le runtime.

Le backend charge ces exports; il ne reentraine pas en live.

Ordre a citer: step1 -> step2 -> step3 -> step4 -> step5 -> step6 -> step6b -> step6c -> step7

Des mesures brutes vers une note de sante

Une feature = un indicateur calcule a partir des mesures brutes

Au depart, on a surtout vibration, puissance, temperature et humidite.

Le pipeline calcule des indicateurs intermediaires, par exemple une moyenne, une variation ou une duree de montee.

Ces indicateurs servent a reconnaitre un comportement

Preuve 3 - Variables construites pour le modele

Les données brutes deviennent des features explicites avant l'apprentissage

```
prediteq_ml/steps/step2_preprocess.py
150 feat_df = pd.DataFrame({
151     'trajectory_id': df['trajectory_id'].values,
152     'profile_id': df['profile_id'].values,
153     't_seconds': t,
154     'simulated_h1': df['simulated_h1'].values,
155     'phase': phase,
156 })
157 return feat_df
158
159 # --- Normalisation Z-score ---
160
161 FEATURE_COLS = [
162     'rms_rms', 'dms_dt', 'rms_variability',
163     'p_mean_kw', 'p_rms_kw', 'dp_dt',
164     'e_cycle_kwh', 'duration_ratio',
165     't_mean_c', 'dt_dt', 'hr_std', 'corr_t_p'
166 ]
167
168 Capture genere automatiquement depuis le code source reel du depot PrediTeq
```

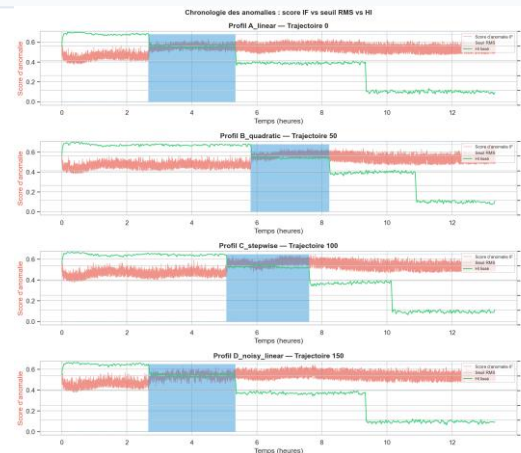
prediteq_ml/steps/step2_preprocess.py

Preuve 4 - Score d'anomalie hybride

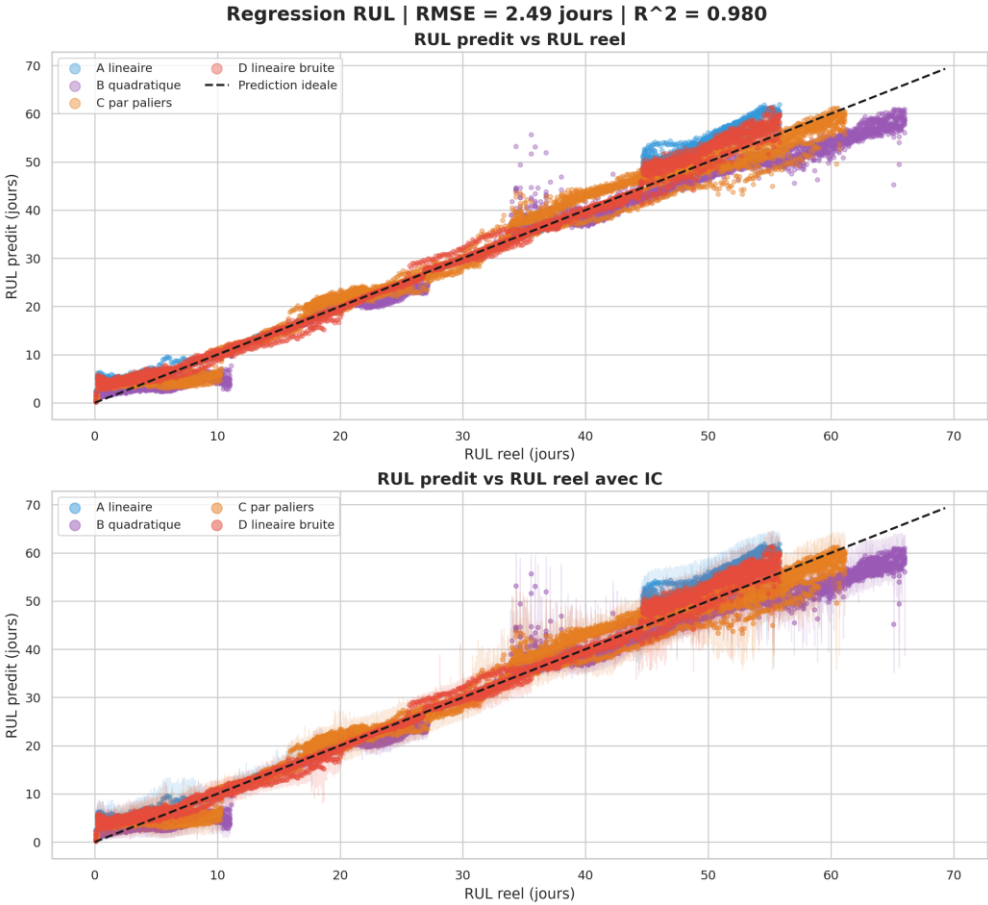
Le score final combine Isolation Forest et vibration RMS avec un poids explicite

```
prediteq_ml/score.py
180 # Quand on TF garde un rôle d'appui pour la dérive multivariée.
181 # Valeur retenue apres calibration train-only sur la correlation entre
182 # MI lisse et simulated_h1 (pas sur le holdout final).
183 MI_BIAS_ALPHA = 0.2
184
185 prediteq_ml/steps/step3_isolation_forest.py
186 sa_min = float(sa_train.quantile(0.01))
187 sa_max = float(sa_train.quantile(0.99))
188 sa_norm = np.clip(scores_off['score_anomaly'].values - sa_min, (sa_max - sa_min) * 0.8, 1.0)
189
190 r1_train = score_off[train_mask, 'rms_score']
191 r1_min = float(r1_train.quantile(0.01))
192 r1_max = float(r1_train.quantile(0.99))
193 r1_norm = np.clip(rms_score - r1_min, (r1_max - r1_min) * 0.8, 1.0)
194
195 scores_off['r1_norm'] = r1_norm
196 scores_off['rms_norm'] = r1_norm
197
198 # 3. Score hybride pondéré : alpha * IF_norm + (1-alpha) * RMS_norm
199 # a été recalculé sur le train pour maximiser la fidélité du MI lisse.
200 alpha = MI_BIAS_ALPHA
201 scores_off['hybrid_score'] = alpha * sa_norm + (1 - alpha) * r1_norm
202 print(f"Hybrid alpha={alpha} IF={alpha * 0.5} RMS={1-alpha * 0.5}")
203
204 Capture genere automatiquement depuis le code source reel du depot PrediTeq
```

prediteq_ml/steps/step3_isolation_forest.py



RUL = temps restant avant qu'une action devienne nécessaire



Le RUL répond a une question simple: combien de temps reste-t-il avant qu'il faille agir ?

Il est construit a partir du Health Index observe par le systeme.

Le franchissement persistant évite les faux changements dus au bruit.

Le but est d'aider a planifier, pas seulement a

2.49 j

erreur moy. sur test
separe

0.982

stabilite par
trajectoires

Lead time moyen avant critique: 62.8 jours eq UI.

Preuve 5 - Cible RUL construite depuis hi smooth

[illegible]

```

176  print(f"GroupFold : chaque fold garde des trajectories entieres en validation.")
177  @staticmethod
178  def split_train_test(train_data, test_data, n_splits):
179      """
180      if train_data is a pandas dataframe, it will split
181      """
182      print(f" [[warn]] Pas assez de groupes pour GroupFold n={n_splits}")
183      return []
184
185  def group_fold_split(split_size, split_size_val):
186      fold_scores = []
187      for fold_idx, (tr_idx, va_idx) in enumerate(zip(split_size(x), group_idx), 1):

```

Comment nous avons verifie les resultats

Plusieurs controles pour eviter l'effet 'ca marche seulement chez nous'

0.964

qualite detection (F1)

0.980

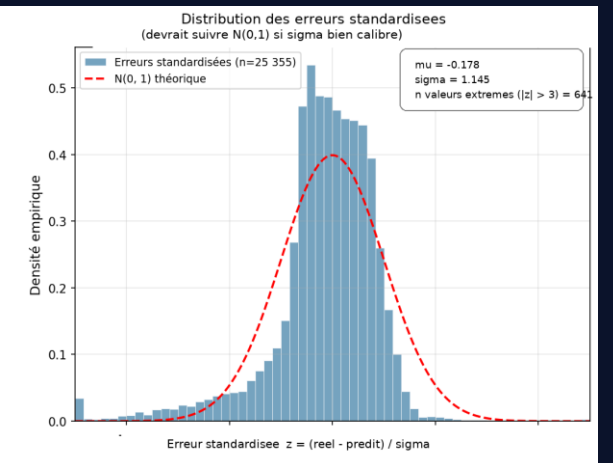
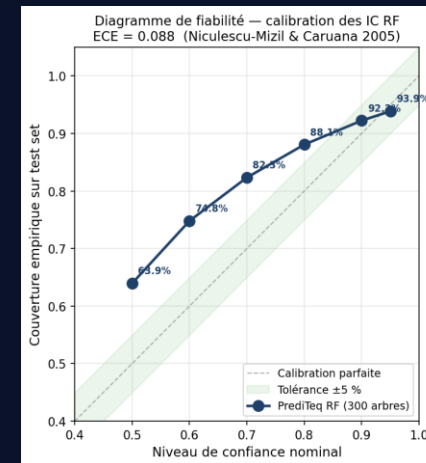
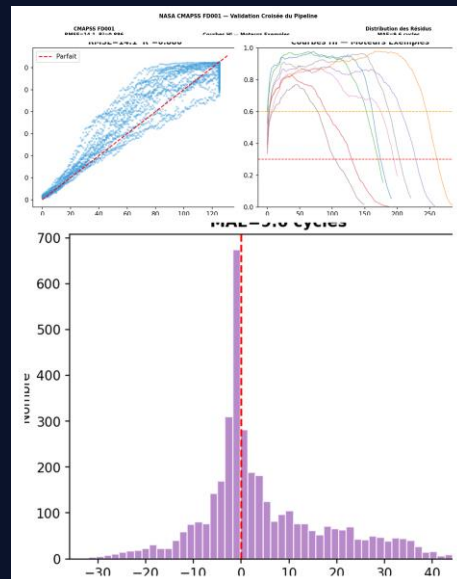
qualite prediction RUL
(R2)

2.32 j

erreur moyenne
validation

0.886 | 14.1

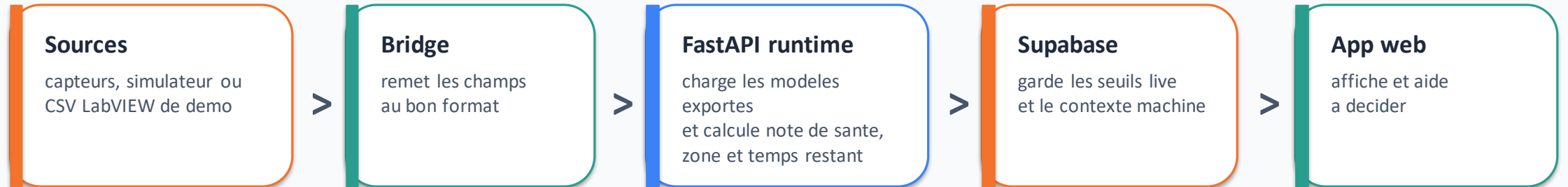
test externe NASA



Pas besoin de retenir tous les sigles: l'idée est que la chaîne reste stable sur un test séparé, sur des trajectoires entières et sur un benchmark externe NASA.

Du signal a l'application

Que se passe-t-il entre une mesure et l'ecran du technicien ?

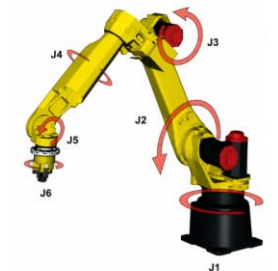
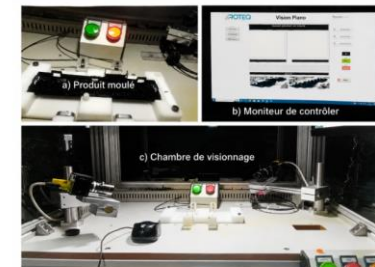


Point cle

Le backend n'entraine rien. Il applique les modeles deja exportes depuis prediteq_ml/models.

Pourquoi c'est rassurant

La meme logique peut etre relue dans les fichiers ML offline puis retrouvee dans le runtime.



Ce que voit l'utilisateur

Le produit final transforme les calculs en decisions lisibles

Dashboard

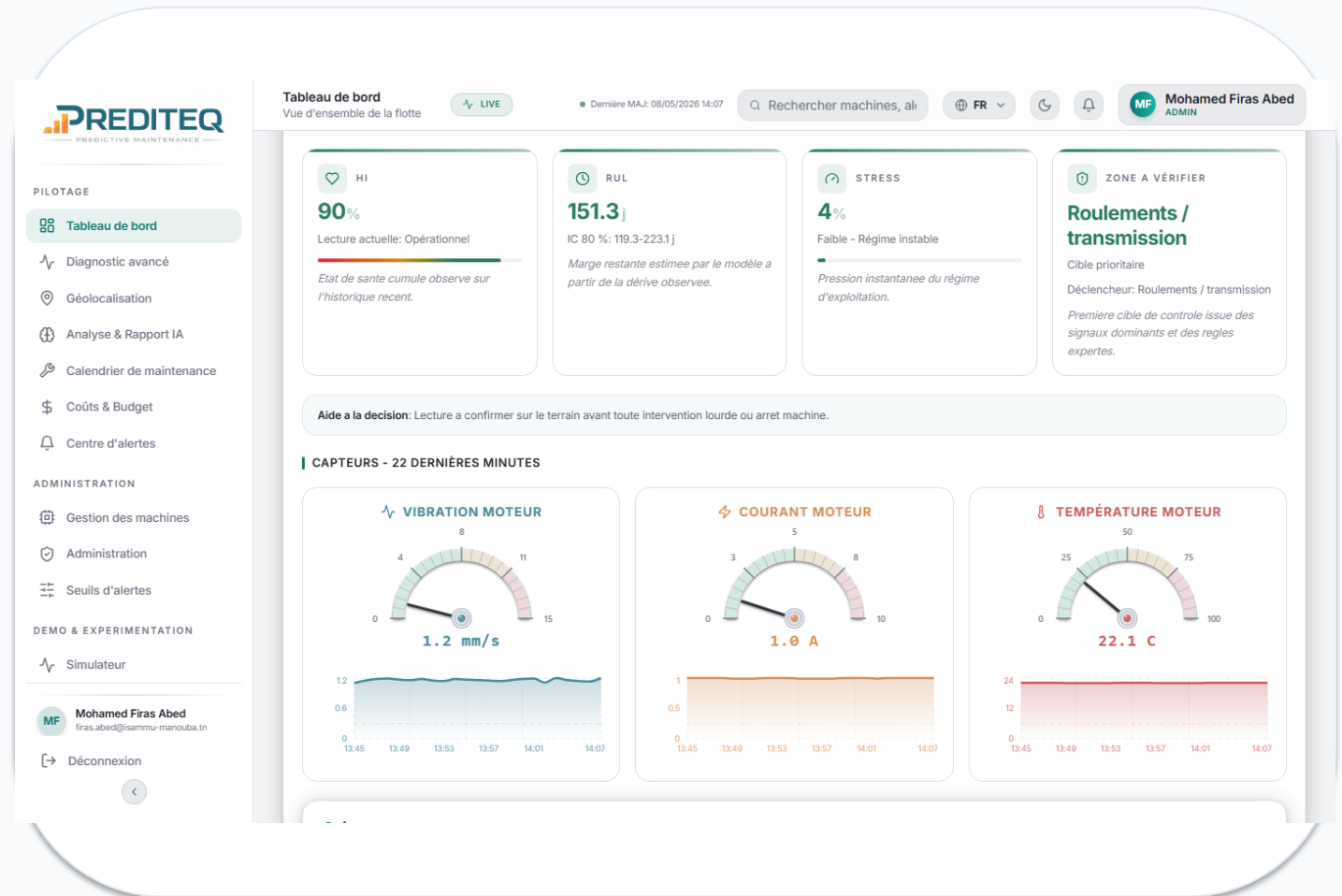
Vue globale de la machine, des alertes et du contexte de fonctionnement.

Diagnostics / RUL

Voir la note de sante, la zone courante et le temps restant avant criticite.

Planner / Maintenance / Costs

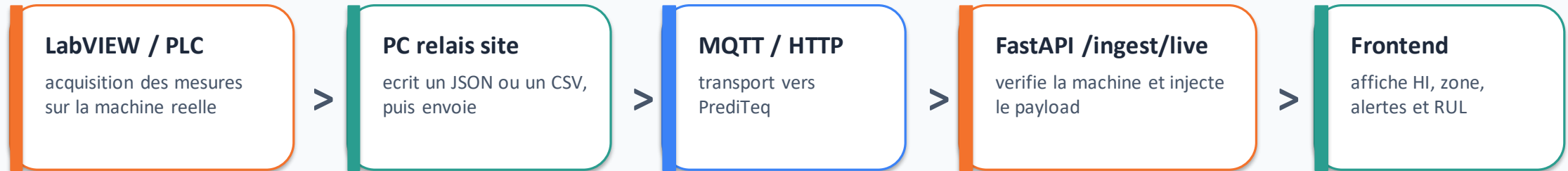
Prioriser l'action, planifier et comprendre l'impact economique.



En demo, ouvrir: Dashboard -> Diagnostics -> Planner -> Maintenance / Costs

Integration machine reelle / PLC / LabVIEW

ARO-01 suit déjà le vrai chemin live; la source finale remplacera surtout le bloc d'acquisition



4 mesures minimales

rms_mms, power_kw, temp_c et humidity_rh.
current_a, load_kg et status restent utiles mais
optionnels.

Ce qui change plus tard

Aujourd'hui, la soutenance utilise une source CSV LabVIEW de demo. Demain, la vraie acquisition PLC/LabVIEW remplacera seulement cette source.



```
prediteq_api/routers/live_ingest.py + prediteq_api/RELAY_PC_SETUP_SIMPLE.md
```

ARO-01 n'est pas une 4e machine simulee : meme moteur runtime, sans overrides simulateur.

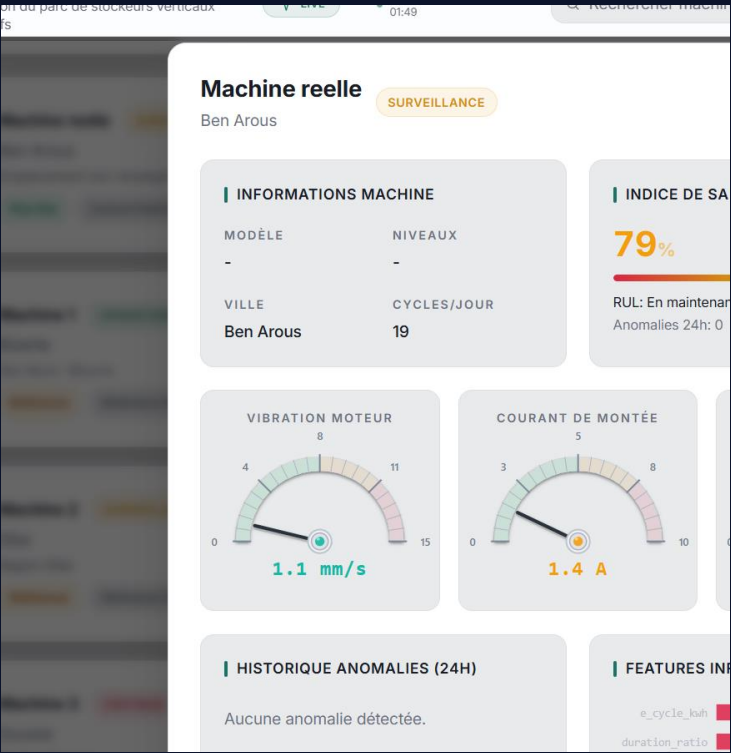
Demonstration live facile a suivre

ARO-01 est d'abord preparee, puis le flux CSV live continue sur la meme machine

bootstrap ARO-01 -> CSV LabVIEW de demonstration -> mqtt_bridge_sender.py -> FastAPI -> application web

```
Starting PrediTeq CSV bridge...
Source locale -> normalisation CSV -> MQTT prive
Machine id: ARO-01
Broker securise en TLS 8883
Topic: prediteq/ARO-01/sensors
Transport: mqtt
Machine id: ARO-01
Target: prediteq/ARO-01/sensors
[2026-05-13T23:43:04.003535+00:00] sent prediteq/ARO-01/sensors rms=1.315 power=1.129 temp=27.0 hum=56.7
[2026-05-13T23:43:05.103632+00:00] sent prediteq/ARO-01/sensors rms=1.374 power=1.129 temp=27.0 hum=58.1
[2026-05-13T23:43:06.184581+00:00] sent prediteq/ARO-01/sensors rms=1.404 power=1.166 temp=26.7 hum=58.4
[2026-05-13T23:43:07.284845+00:00] sent prediteq/ARO-01/sensors rms=1.535 power=1.150 temp=26.3 hum=58.8
[2026-05-13T23:43:08.368216+00:00] sent prediteq/ARO-01/sensors rms=1.512 power=0.987 temp=26.2 hum=59.5
[2026-05-13T23:43:09.450089+00:00] sent prediteq/ARO-01/sensors rms=1.525 power=0.840 temp=25.6 hum=60.1
[2026-05-13T23:43:10.549100+00:00] sent prediteq/ARO-01/sensors rms=1.421 power=0.693 temp=25.1 hum=61.0
[2026-05-13T23:43:11.646307+00:00] sent prediteq/ARO-01/sensors rms=1.291 power=0.583 temp=24.8 hum=60.6
[2026-05-13T23:43:12.744875+00:00] sent prediteq/ARO-01/sensors rms=1.176 power=0.511 temp=24.5 hum=61.0
```

```
PS C:\workspace\prediteq_api>
python -m uvicorn main:app --reload
INFO: Will watch for changes in ['app']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [11856] using WatchFiles
INFO: Started server process [7888]
INFO: Waiting for application startup.
02:49:49 | INFO | prediteq | Starting PrediTeq API ...
02:49:49 | INFO | core.sunahase_client | Sunahase client initialized
02:49:49 | INFO | httpx | GET https://supabase-instance/rest/v1/machines ... "HTTP/2 200 OK"
ml\models
02:49:49 | INFO | ml.loader | Loaded isolation_forest.pkl
02:49:49 | INFO | ml.loader | Loaded random_forest_rul.pkl (300 trees)
02:49:49 | INFO | ml.loader | Loaded scaler_params.json (12 features)
02:49:49 | INFO | ml.loader | Loaded hi_params.json (p5=0.0522, p95=0.8631)
02:49:49 | INFO | ml.loader | Loaded hybrid_params.json (alpha=0.20)
02:49:49 | INFO | ml.engine_manager | Registered 4 machines: ['ARO-01', 'ASC-A1', 'ASC-B2', 'ASC-C3']
02:49:50 | INFO | routers.mqtt | Dedicated bridge account loaded
02:49:50 | INFO | routers.mqtt | MQTT connected - subscribed to prediteq/+sensors
02:49:50 | INFO | routers.mqtt | MQTT connecting to broker TLS 8883 ...
```



1. Source de donnees

2. Calcul backend

3. Resultat dans l'app

Conclusion et prochaines etapes

Ce qui est déjà solide, puis ce qui doit encore être rapproché du terrain

Deja en place

Jeu d'entraînement simulé relié à la logique machine.

Trois machines demo distinctes pour raconter trois régimes d'usage.

Pipeline ML ordonne et mesure sur plusieurs tests.

Backend runtime + application web + intégration live déjà structurée.

Suite logique

Augmenter le volume de données terrain réelles et les annotations.

Remplacer progressivement la source CSV demo par une acquisition industrielle directe.

Affiner encore la calibration et les seuils par contexte machine.

Étendre la généralisation vers d'autres équipements et profils d'usage.

Phrase de fin: PrediTeq relie de façon cohérente la physique machine, le Machine Learning, le runtime IoT et la décision dans une application web.

Backup - jeu d'entrainement et preuves simulation

step1_simulate.py + config.py: ce que nous presentons vient du vrai code

```
Preuve 7 - Jeu d'entrainement simule
200 Trajectoires, 4 profils, 20 cas de charge et vibrations exportées vers trajectories.csv

prediteq_ml/steps/step1_simulate.py

# Etape 1 - Simulation de données
1 Génère N_TRAJECTORIES (+200) trajectoires de dégradation physiquement contraintes.
2 Choix trajectoire à un cas de charge qui détermine l'appel de courant.
3 Sortie : data/raw/trajectories.csv

4
5 Chaîne de dégradation (d'après technicien Aroteq, confirmée par IEC 60834-1 §8.5) :
6 charge f = puissance / courant f = échauffement bobines (loi 1/F) f
7 = vieillissement isolation (Arrhenius) + fatigue palliers (Lundberg-Palmgren)
8 ~ HI ~ RMS Vibratoire T (ISO 10816-3)
9 La variable électrique de régression est le COURANT (400V) et
10 vitesse (1418 RPM) sont constantes d'après la plaque signalétique.
11
12
13 Profils de dégradation (familles standard en PEM, cf. Saeens & Gobel 2008) :
14 A_linear : dégradation uniforme, usure abrasive simple
15 B_quadratic : profil à deux temps (lent puis accéléré), formule HI(t)
16 = 1 - k * (t/T_fall)^2. Inspirée de la propagation de fissure
17 sous-tréfilon (Ind de Paris-Fredgen simplifiée). Remonté
18 en avril 2016 (anciennement ~ B_exponential *) pour refléter
19 exactement la formule mathématique implémentée.
20
21 C_tapeuse : défauts successifs discrets (dents d'engrenage, écaillage)
22 D_noisy_linear : linéaire + bruit mesure (cas réaliste avec capteurs bruyés)
23
24
25 return pd.DataFrame({
26     'trajectory_id': traj_id,
27     'profile': profile,
28     'load_kg': load_kg,
29     't_seconds': t_seconds,
30     'rms_vms': rms_vms,
31     'power_kw': power_kw,
32     'current_a': current_a,
33     'temp_c': temp_c,
34     'humidity_rm': humid_rm,
35     'simulated_hi': hi,
36     'phase': phase,
37 })
38
39
40 if __name__ == '__main__':
41     all_dfs = []
42     traj_id = 0
43     n_each = N_TRAJECTORIES // N_PROFILES * 200/4 = 50 par profil
44
45 # Répartir les cas de charge entre les trajectoires de chaque profil :
46 # 50 trajectoires par profil = 4 profils = 200 au total
47 # 20 cas de charge = 2 trajectoires par cas = 10 cas répétés (50 = 20*2+10)
48 load_list = LOAD_CASES_KG * (n_each // N_LOAD_CASES) + \
49     LOAD_CASES_KG[n_each % N_LOAD_CASES]
50 load_list = sorted(load_list) # ordre déterministe
51
52 for profile in PROFILE_NAMES:
53     loads_for_profile = load_list[n_each]
54     print(f" Simulation (profile) ((n_each) trajectoires, "
55           f"charges: (sorted(set(loads_for_profile)) kg)...")
56     for i in range(n_each):
57         load_kg = loads_for_profile[i]
58         df = simulate_trajectory(traj_id, profile, load_kg)
59         all_dfs.append(df)
60         traj_id += 1
61
62 trajectories = pd.concat(all_dfs, ignore_index=True)
63
64 out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'data', 'raw', 'trajectories.csv')
65 out_path = os.path.abspath(out_path)
66 os.makedirs(os.path.dirname(out_path), exist_ok=True)
67 trajectories.to_csv(out_path, index=False)
68
69 print(f"MDK: Etape 1 terminée - {traj_id} trajectoires, {len(trajectories):,} lignes")
70
71 Capture générée automatiquement depuis le code source réel du dépôt Prediteq.
```

prediteq_ml/steps/step1_simulate.py

```
Preuve 1 - Constantes moteur, charge et cycle
Capture du code source réel de la configuration de simulation.

prediteq_ml/config.py

11 # Note : le technicien a dit ~ 380V ~ à l'oral - ancien non pour 400V (harmonisation UE).
12 # La plaque signalétique indique 400V. On utilise la valeur de la plaque.
13 MOTOR_SPEED_RPM = 1418 # Constante - ne varie pas avec la charge
14 MOTOR_POWER_KW = 2.2 # Puissance utile sur l'arbre (mécanique)
15 MOTOR_VOLTAGE_V = 400 # Tension triphasée (couplage V, d'après plaque)
16 MOTOR_COSPHI = 0.88 # Facteur de puissance (d'après plaque)
17 REDUCER_RATIO = 1/40
18
19 # Géométrie machine
20 N_FLOORS = 19
21 LOAD_PER_FLOOR_KG = 15 # Max 15 kg par étage (d'après technicien)
22 LOAD_MAX_KG = 19 * 15 # ~ 285 kg ~ 19 étages entièrement chargés
23 LOAD_NOMINAL_KG = 10 * 15 # ~ 150 kg ~ demi-charge (fonctionnement typique)
24
25 # Timing du cycle (secondes)
26 T_ASCENT_S = 12
27 T_DESCENT_S = 12
28 T_PAUSE_S = 20
29 T_CYCLE_S = 44
30
31
32 # N_TRAJECTORIES = 200 : dimensionnement statistique
33 # ~ 50 trajectoires par profil * 4 profils couvre toute la plage de charge
34 # (20 cas @ 285 kg, répétés ~1.5x) et stabilise la variance CV
35 # ~ ordre de grandeur similaire à CHOPS F0801 (100 unités) et FEMTO-ST PROMOSTIA
36 # (~17 moments run-to-failure), Saeens & Gobel 2008
37 N_TRAJECTORIES = 200
38
39 Capture générée automatiquement depuis le code source réel du dépôt Prediteq.
```

prediteq_ml/config.py

```
Preuve 2 - Puissance, phase et courant
Le cycle ascent / descent / pause et la derive électrique sont simulés dans le code.

prediteq_ml/steps/step1_simulate.py

140 for i in range(n):
141     pt = phase[i]
142     h = hi[i]
143     if pt < T_ASCENT_S:
144         phase[i] = 'ascent'
145         # Puissance = base (friction) + composante charge + composante dégradation
146         # P = P_vide + P_plage_charge + (charge/max) * P_plage_deg * (1 - HI)
147         p_load = P_ASCENT_EMPTY_KW + P_ASCENT_LOAD_RANGE * load_ratio
148         p_deg = P_ASCENT_DEG_RANGE * (1 - h)
149         power[i] = np.clip(p_load + p_deg, P_ASCENT_EMPTY_KW, P_ASCENT_DEG_KW)
150         elif pt < T_DESCENT_S + T_DESCENT_S:
151             phase[i] = 'descent'
152             power[i] = P_DESCENT_KW
153         else:
154             phase[i] = 'pause'
155             power[i] = P_PAUSE_KW
156
157 # Bruit capteur (PAC200 - fiche technique section 3.1) ; noise_mult
158 # pour capteurs dégradés (profil D, simule EMI variableur).
159 power += np.random.normal(0, NOISE_PAC200 * P_ASCENT_NOM_KW * noise_mult, size=n)
160 power = np.clip(power, 0.0, 3.0)
161
162 # Courant : I = P / (√3 * V * cosφ) ~ tension constante à 400V
163 current = (power * 1000) / (MOTOR_SPEED_V_COSPHI * KH * N, puis / dénominateur
164
165 Capture générée automatiquement depuis le code source réel du dépôt Prediteq.
```

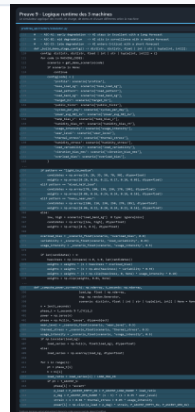
prediteq_ml/steps/step1_simulate.py

Backup - preuves des 3 machines de demo

demo_scenarios.py + simulator.py: charges, stress, usure et cibles HI



prediteq_api/demo_scenarios.py



prediteq_api/routers/simulator.py

Backup - preuves ML actuelles

step2, step3, step4 et step5: les vraies briques de la chaine ML

Preuve 3 - Variables construites pour le modele

Les données brutes deviennent des features explicites avant l'apprentissage

prediteq_ml/steps/step2_preprocess.py

```
145 feat_df = pd.DataFrame({
146     'trajectory_id': df['trajectory_id'].values,
147     'profile': df['profile'].values,
148     't_seconds': t,
149     'simulated_hi': df['simulated_hi'].values,
150     'phase': phase,
151     **feats,
152 })
153 return feat_df
154
155 # --- Normalisation Z-score ---
156
157 FEATURE_COLS = [
158     'rms_rms', 'drms_dt', 'rms_variability',
159     'p_mean_kw', 'p_rms_kw', 'dp_dt',
160     'e_cycle_kwh', 'duration_ratio',
161     't_mean_c', 'dt_dt', 'hr_std', 'corr_t_p'
```

Capture generée automatiquement depuis le code source réel du depot PrediTeq.

Preuve 4 - Score d'anomalie hybride

Le score final combine Isolation Forest et vibration RMS avec un poids explicite

prediteq_ml/config.py

```
188 # tandis que IF garde un rôle d'appoint pour la dérive multivariée.
189 # Valeur retenue après calibration train-only sur la corrélation entre
190 # HI lissé et simulated_hi (pas sur le holdout final).
191 HYBRID_ALPHA = 0.2
192
193 prediteq_ml/steps/step3_isolation_forest.py
194
195 sa_min = float(sa_train.quantile(0.01))
196 sa_max = float(sa_train.quantile(0.99))
197 sa_norm = np.clip((scores_df['score_anomaly'].values - sa_min) / (sa_max - sa_min + 1e-8), 0.0, 1.0)
198
199 r2_train = scores_df.loc[train_mask, 'rms_score']
200 r2_min = float(r2_train.quantile(0.01))
201 r2_max = float(r2_train.quantile(0.99))
202 r2_norm = np.clip((rms_score - r2_min) / (r2_max - r2_min + 1e-8), 0.0, 1.0)
203
204 scores_df['if_norm'] = sa_norm
205 scores_df['rms_norm'] = r2_norm
206
207 # 3. Score hybride pondéré : alpha * IF_norm + (1-alpha) * RMS_norm
208 # a été recalibré sur le train pour maximiser la fidélité du HI lissé.
209 alpha = HYBRID_ALPHA
210 scores_df['hybrid_score'] = alpha * sa_norm + (1 - alpha) * r2_norm
211 print(f" Hybrid alpha={alpha} (If={alpha:.0%}, RMS={1-alpha:.0%})")
```

Capture generée automatiquement depuis le code source réel du depot PrediTeq.

Preuve 5 - Cible RUL construite depuis hi_smooth

La prediction de durée de vie utilise le HI observé et des retours de tendance

prediteq_ml/steps/step5_rul_model.py

```
123 hi_obs = merged[hi_smooth].values
124
125 # Franchissement confirmé : N points consécutifs sous le seuil critique
126 t_fail_obs = find_overhead_crossing(
127     hi_obs, HI_CRITICAL, HI_CROSSING_PERSISTENCE
128 )
129
130 if t_fail_obs is None:
131     h_scaled_hi_cross += 1
132     continue
133
134 t_fail_obs = merged.loc[t_fail_obs, 't_seconds'] / 60.0
135
136 # Générer des échantillons RUL du début (après la fenêtre lookback)
137 # "Jouer" à l'échelle de défaillance.
138 for i in range(t_lookback_obs, t_fail_obs):
139     window = hi_obs[max(0, i - RUL_LOOKBACK_OBS):i]
140     if len(window) < RUL_LOOKBACK_OBS:
141         continue
142
143     t_obs_min = merged.loc[i, 't_seconds'] / 60.0
144     rul_obs = t_fail_obs - t_obs_min
145     if rul_obs < 0:
146         h_scaled_early += 1
147         continue
148
149 sensor_feats = merged.loc[i, RUL_COLS].values.astype(float)
150
151 hi_obs = hi_obs[i]
152 hi_mean = np.mean(window)
153 hi_std = np.std(window)
154 hi_min = np.min(window)
155 hi_max = np.max(window)
156 hi_slope = np.polyfit(np.arange(len(window)), window, 1)[0]
157
158 row = np.concatenate([sensor_feats, [hi_obs, hi_mean, hi_std, hi_min, hi_max, hi_slope]])
```

Capture generée automatiquement depuis le code source réel du depot PrediTeq.

Preuve 6 - Validation GroupKFold par trajectoire

Le split de validation laisse des trajectoires entières hors apprentissage

prediteq_ml/steps/step5_rul_model.py

```
175 """GroupKFold : chaque fold garde des trajectoires entières en validation.
176 Évite la fuite intra-trajectoire (Kuhn & Johnson 2013, §4.3)."""
177 if len(X) == 0 or len(np.unique(groups)) < n_splits:
178     print(f" [warn] Pas assez de groupes pour GroupKFold k={n_splits}")
179     return []
180
181 gkf = GroupKFold(n_splits=n_splits)
182 fold_scores = []
183 for fold_idx, (tr_idx, va_idx) in enumerate(gkf.split(X, y, groups), 1):
```

Capture generée automatiquement depuis le code source réel du depot PrediTeq.

Contrat HTTP/MQTT, puis commandes minimales pour la demo locale

Proceding expose un point d'entrée lié avec 4 mesures intermédiaires et un chemin sous-PC document

```
cd prediteq_api\npython scripts/generate_labview_demo_csv.py --  
machine-id ARO-01 --scenario surveillance\npython  
scripts/setup_real_machine_demo.py --machine-id ARO-01 --name 'Machine  
reelle' --scenario surveillance\npython  
scripts/replay_labview_demo_csv.py --input scripts/sample_data/ARO-  
01_labview_demo_template.csv --output C:\\labview\\prediteq_log.csv --  
interval 1.0\npython scripts/mqtt_bridge_sender.py --transport mqtt --  
mode csv-last-row --machine-id ARO-01 --csv-path  
C:\\labview\\prediteq_log.csv
```

Backup - fichiers source de verite

Ou montrer le code actuel en moins de 30 secondes

Jeu d'entrainement

```
prediteq_ml/config.py  
prediteq_ml/steps/step1_simulate.py
```

3 machines demo

```
prediteq_api/demo_scenarios.py  
prediteq_api/routers/simulator.py
```

ML / resultats

```
prediteq_ml/steps/step2_preprocess.py  
prediteq_ml/steps/step3_isolation_forest.py  
prediteq_ml/steps/step4_health_index.py  
prediteq_ml/steps/step5_rul_model.py
```

Runtime backend

```
prediteq_api/ml/engine_manager.py  
prediteq_api/routers/diagnostics_rul.py  
prediteq_api/routers/live_ingest.py
```

Flux live / PC relais

```
prediteq_api/scripts/setup_real_machine_demo.py  
prediteq_api/scripts/generate_labview_demo_csv.py  
prediteq_api/scripts/mqtt_bridge_sender.py  
prediteq_api/LABVIEW_CSV_BRIDGE_DEMO.md
```